Universität Bonn

Mitschrift zur Vorlesung

Randomisierte und Approximative Algorithmen

gehalten von

Prof. Dr. Thomas Kesselheim, Prof. Dr. Heiko Röglin

geTEXed von

Daniel Emse

Winter 2025/26

Anmerkungen und Korrekturen

Falls du Anmerkungen/Korrekturen hast, melde dich bei mir (`daniel.emse@uni-bonn.de`).

Inhaltsverzeichnis

Vorlesung **1**

I Randomisierte und Approximative Algorithmen **2**

1 Einleitung **3**

2 Greedy-Algorithmen **4**
2.1 Vertex Cover 4
2.2 Set Cover 5
2.3 Scheduling auf identischen Maschinen 6
2.4 Rucksack-Problem 8

Anhang **A**

Liste der Definitionen **B**

Liste der Aussagen **C**

Liste der Algorithmen **D**

Index **E**

Vorlesung

Teil I

Randomisierte und Approximative Algorithmen

Kapitel 1

Einleitung

Definition 1.1: Optimierungsproblem, Approximationsalgorithmus

Ein *Optimierungsproblem* Π besteht aus

- einer Meinge von Instanzen $\mathcal{I}_\Pi$
- für jede Instanz $I \in \mathcal{I}_\Pi$
  - einer Menge $\mathcal{S}_I$ von Lösungen
  - einer Zielfunktion $f_I : \mathcal{S}_I \to \mathbb{R}_{\geq 0}$.

Ein *Approximationsalgorithmus* nimmt I als Eingabe und berechnet $A(I) \in \mathcal{S}_I$ (in polynomieller Zeit).
Sei $w_A(I) = f_I(A(I))$. Ein Approximationsalgorithmus erreicht Approximationsfaktor r, falls

$$w_A(I) \leq r \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Minimierungsprobleme)},$$

bzw.

$$w_A(I) \geq \frac{1}{r} \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Maximierungsprobleme)}.$$

Kapitel 2

Greedy-Algorithmen

2.1 Vertex Cover

[14.10.2025, Vorlesung 1]

Problem 2.1: Vertex Cover

Input: Ungerichteter Graph $G = (V, E)$
Output: Alle $V' \subseteq V$, sodass $\forall e = (u, v) \in E$ gilt $u \in V'$ oder $v \in V'$
Objective: Minimiere $|V'|$

Definition 2.2: Matching

Ein *Matching* $M \subseteq E$ eines Graphen $G = (V, E)$ ist eine Teilmenge der Kanten, sodass es keinen Knoten gibt, der zu mehr als einer Kante aus M inzident ist.
Es heißt *inklusionsmaximal*, wenn für alle Kanten $e \in E \setminus M$ gilt: $M \cup \{e\}$ ist kein Matching.

Algorithmus 2.3 MATCHING-VC

Input: ungerichteter Graph $G = (V, E)$
Output: Vertex Cover $V' \subseteq V$ mit $|V'| \leq 2 \cdot \mathrm{OPT}(G)$

1: ▷ *Berechne inklusionsmaximales Matching $M \subseteq E$:* ◁
2: $M := \emptyset$
3: **for all** $e = \{u, v\} \in E$ **do**
4: **if** weder u noch v ist inzident zu einer Kante in M **then**
5: $M := M \cup \{e\}$
6: $V(M) := \{v \in V \mid v \text{ ist inzident zu einer Kante in } M\}$
7: **return** $V(M)$

Satz 2.4: Matching-VC

Matching-VC hat Laufzeit $\mathcal{O}(|V| + |E|)$ für Graphen in Adjazenzlistendarstellung. Er berechnet ein Vertex Cover der Größe höchstens $2 \cdot \mathrm{OPT}(G)$, wobei $\mathrm{OPT}(G)$ die Größe des kleinsten Vertex Covers ist.

Beweis. **Laufzeit:** In Schritt 1 wird jede Kante genau einmal betrachtet, $\mathcal{O}(|V| + |E|)$.

In Schritt 2 wird jeder Knoten höchstens einmal betrachtet, $\mathcal{O}(|V|)$. Gemeinsam ergibt sich $\mathcal{O}(|V| + |E|)$.

Korrektheit: Angenommen, $e = (u, v) \in E$ wurde nicht von $V(M)$ abgedeckt. Das heißt, es wären weder $u \in V(M)$ noch $v \in V(M)$. Dann wäre $M \cup \{e\}$ ein Matching und e würde in Schritt 1 zu M hinzugefügt werden. Widerspruch zur Inklusionsmaximalität von M. Also ist $V(M)$ ein Vertex Cover.

Approximation: Sei V^* ein minimales Vertex Cover. V^* muss alle Kanten $e \in E$ abdecken, also auch alle $e \in M$.

Außerdem haben in M keine zwei Kanten denselben Endpunkt. Deshalb ist $|M| \leq |V^*|$.

Für unsere Lösung gilt $|V(M)| = 2 \cdot |M| \leq 2 \cdot \mathrm{OPT}(G)$.

□

2.2 Set Cover

Problem 2.5: Set Cover

Input: Menge S mit n Elementen, n Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^m S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: Alle $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$
Objective: Minimiere $\sum_{i \in A} c_i$

Algorithmus 2.6 GREEDY-SC

Input: Grundmenge S mit n Elementen, n Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^m S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$ und $\sum_{i \in A} c_i$ minimal

1: $A := \emptyset, C := \emptyset$
2: **while** $C \neq S$ **do**
3: Wähle S_i mit minimalen relativen Kosten $r_i(C) = \frac{c_i}{|S_i \setminus C|}$
4: Setze $\mathrm{price}(x) := r_i(C)$ für alle $x \in S_i \setminus C$
5: $A := A \cup \{i\}$
6: $C := C \cup S_i$
7: **return** A

Satz 2.7: Greedy-SC

GREEDY-SC ist ein H_n-Approximationsalgorithmus für Set Cover, wobei

$$H_n = \sum_{i=1}^{n} \frac{1}{i}$$

die n-te harmonische Zahl ist.

Untere Schranke für Greedy-SC: Sei $\varepsilon > 0$ sehr klein. Betrachte n Knoten und $m = n + 1$ Mengen, wobei $S_i = \{i\}$ mit Kosten $\frac{1}{n+1-i}$ ist (für $i \in S$) und $S_{n+1} = S$ mit Kosten $1 + \varepsilon$. Dann wählt GREEDY-SC wählt alle Einzelnen, OPT die gesamte Menge.

[14.10.2025, Vorlesung 1]
[15.10.2025, Vorlesung 2]

Beweis. Beobachtung: Es gilt

$$c(A) = \sum_{x \in S} \mathrm{price}(x).$$

Sei $x_1, \ldots, x_n$ die Reihenfolge der Elemente in S, wie sie vom GREEDY-SC abgedeckt werden (Beim Abdecken von mehreren Elementen in gleicher Iteration wähle beliebige Reihenfolge).

Lemma 2.8

Für alle $k \in \{1, \dots, n\}$ gilt

$$\text{price}(x_k) \leq \frac{\text{OPT}}{n-k+1}.$$

Beweis: Lemma 2.8. Betrachte ein beliebiges $k \in \{1, \dots, n\}$. Sei $i \in \{1, \dots, m\}$ der Index der Menge S_i, mit der x_k erstmalig abgedeckt wird. Bezeichne A die Auswahl der Mengen, *bevor* x_k abgedeckt wird.

Sei $C := \bigcup_{i \in A} S_i$. Es gilt $|C| \leq k-1$. Eine optimale Auswahl A^* von Mengen deckt ganz S ab, insbesondere auch $S \setminus C$, wobei $|S \setminus C| \geq n-k+1$.

Es gilt

$$\begin{aligned}\text{OPT} = \sum_{j \in A^*} c_j &\geq \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} c_j = \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \underbrace{\frac{c_j}{|S_j \setminus C|}}_{= r_j(C) \geq r_i(C)} \\ &\geq r_i(C) \underbrace{\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C|}_{\geq |S \setminus C|} \geq \underbrace{r_i(C)}_{= \text{price}(x_k)} \cdot (n-k+1)\end{aligned}$$

wobei $r_j(C) \geq r_i(C)$ wegen der Greedy-Auswahl und

$$\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \geq |S \setminus C|,$$

weil A^* ganz $S \setminus C$ abdeckt. □

Nun gilt mit Lemma 2.8

$$c(A) = \sum_{k=1}^{n} \text{price}(x_k) \leq \sum_{k=1}^{n} \frac{\text{OPT}}{n-k+1} = \text{OPT} \sum_{i=1}^{n} \frac{1}{i} = \text{OPT} \cdot H_n$$

und Satz 2.7 ist gezeigt. □

2.3 Scheduling auf identischen Maschinen

Problem 2.9: Scheduling auf identischen Maschinen

Input: Menge $J = \{1, \dots, n\}$ von Jobs, Menge $M = \{1, \dots, m\}$ von Maschinen, Größen $p_j > 0$ für jeden Job $j \in J$

Output: Schedule $\pi : J \to M$, Last von Maschine i in Schedule π: $L_i(\pi) = \sum_{\substack{j \in J \\ \pi(j) = i}} p_j$, Makespan $C(\pi) = \max_{i \in M} L_i(\pi)$

Objective: Finde Schedule π mit kleinstem Makespan

Algorithmus 2.10 LEASTLOADED

Input: Siehe Problem 2.9

Output: Siehe Problem 2.9

1: Betrachte Jobs in Reihenfolge $1, \dots, n$
2: Weise jedem Job der Maschine mit bislang kleinster Last zu

Satz 2.11: LeastLoaded

LeastLoaded ist ein $(2-\frac{1}{m})$-Approximationsalgorithmus für Scheduling auf identischen Maschinen.

Beweis. Sei π^* ein optimaler Schedule. Dann ist

$$C(\pi^*) \geq \frac{1}{m} \sum_{j \in J} p_j,$$

denn

$$C(\pi^*) = \max_{i \in M} \sum_{\substack{j \in J \\ \pi(j)=i}} p_j \geq \frac{1}{m} \sum_{i \in M} \sum_{\substack{j \in J \\ \pi(j)=i}} = \frac{1}{m} \sum_{j \in J} p_j.$$

Außerdem gilt $C(\pi^*) \geq \max_{j \in J} p_j$.

Sei π der Schedule, der LeastLoaded berechnet. Sei i die Maschine mit größter Last. Sei j der Job, der zuletzt zu i hinzugefügt wurde. Zu diesem Zeitpunkt war Maschine i diejenige mit der geringsten Last. Ihre Last war höchstens $\frac{1}{m}\sum_{k=1}^{j-1} p_k$. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m} \underbrace{\sum_{k=1}^{j-1} p_k}_{\leq \sum_{k \in J \setminus \{j\}} p_k} + p_j \leq \underbrace{\frac{1}{m} \sum_{k \in J} p_k}_{\leq C(\pi^*)} + \left(1 - \frac{1}{m}\right) \underbrace{p_j}_{\leq C(\pi^*)} \leq \left(2 - \frac{1}{m}\right) C(\pi^*)$$

und die Abschätzung ist gezeigt. □

Bemerkung 2.12. $2 - \frac{1}{m}$ ist eine untere Schranke für LeastLoaded.

Beweis. Sei $n = m(m-1)+1$ und $p_1 = \cdots = p_{m(m-1)} = 1$ und $p_{m(m-1)+1} = m$.

LeastLoaded führt auf jeder Maschine $m-1$ Jobs der Größe 1 durch. Auf einer Maschine wird zusätzlich ein Job der Größe m ausgeführt. Somit ist der Makespan $m-1+m = 2m-1$.

Eine optimale Belegung führt auf $m-1$ Maschinen je m Jobs der Größe 1 aus und auf der letzten Maschine einen Job der Größe m, wir haben also einen Makespan von m, und der Faktor stimmt. □

Algorithmus 2.13 LongestProcessingTime (LPT)

Input: Siehe Problem 2.9
Output: Siehe Problem 2.9

1: Sortiere Jobs so, dass $p_1 \geq p_2 \geq \cdots \geq p_n$.
2: Führe LeastLoaded auf den sortierten Jobs aus.

Satz 2.14: LPT

LongestProcessingTime ist ein $\frac{4}{3}$-Approximationsalgorithmus.

Beweis. Angenommen, es gäbe eine Instanz, sodass LPT einen Makespan von mehr als $\frac{4}{3} \cdot \mathrm{OPT}$ erreicht. Betrachte unter diesen Instanzen eine mit geringster Anzahl Jobs.

Sei i eine Maschine mit höchster Last in LPT-Schedule π. Sei j der letzte Job, der Maschine i zugewiesen wird. Es ist $j = n$, da sonst $p_1, \ldots, p_j$ ein Gegenbeispiel mit weniger Jobs wäre. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m} \sum_{k=1}^{n-1} p_k + p_n \leq \mathrm{OPT} + p_n.$$

Aus $C(\pi) > \frac{4}{3}\,\mathrm{OPT}$ folgt $p_n > \frac{1}{3}\,\mathrm{OPT}$. Da $p_1 \geq \cdots \geq p_n$ folgt $p_j > \frac{1}{3}\,\mathrm{OPT}$ für alle $j \in J$. Also erhält im optimalen Schedule jede Maschine höchstens zwei Jobs.

Sei k die Anzahl Jobs mit Größe $\frac{2}{3}\,\mathrm{OPT}$. Im optimalen Schedule sind k Jobs alleine auf einer Maschine und $n-k$ Jobs höchstens zu zweit.

Fall 1: $k = n$**.** Also ist $n \leq m$, Job n wird also einer leeren Maschine zugewiesen.

Fall 2: $k < n$**.** Zum Zeitpunkt bevor Job n hinzugefügt wird, hat LPT mindestens eine Maschine mit höchstens einem Job und dieser hat Größe höchstens $\frac{2}{3}\,\mathrm{OPT}$. Wegen $p_n \leq \frac{2}{3}\,\mathrm{OPT}$ ist die Last dieser Maschine im Anschluss höchstens $\frac{4}{3}\,\mathrm{OPT}$.

Beides ist ein Widerspruch zur Annahme. ↯ □

Bemerkung 2.15. $\frac{4}{3}$ ist eine untere Schranke für LongestProcessingTime.

Beweis. Übungsaufgabe. □

Wir werden später sehen, dass für alle $\varepsilon > 0$ ein $(1+\varepsilon)$-Approximationsalgorithmus existiert.

[15.10.2025, Vorlesung 2]
[21.10.2025, Vorlesung 3]

2.4 Rucksack-Problem

[21.10.2025, Vorlesung 3]
[22.10.2025, Vorlesung 4]

Anhang

Liste der Definitionen

1.1 Optimierungsproblem, Approximationsalgorithmus 3
2.1 Vertex Cover 4
2.2 Matching 4
2.5 Set Cover 5
2.9 Scheduling auf identischen Maschinen 6

Liste der Aussagen

2.4 Matching-VC 4
2.7 Greedy-SC 5
2.8 6
2.11 LeastLoaded 7
2.14 LPT 7

Liste der Algorithmen

2.3 Matching-VC 4
2.6 Greedy-SC 5
2.10 LeastLoaded 6
2.13 LongestProcessingTime (LPT) 7

Index

Greedy-SC, 5

LeastLoaded, 6
LongestProcessingTime, 7
LPT, 7

Matching, 4
    inklusionsmaximal, 4
Matching-VC, 4

Scheduling, 6
    auf identischen Maschinen, 6
Set Cover, 5

Vertex Cover, 4